

Техническое задание

Проект «Парк-и-Гоу» – сервис бронирования гостевых парковочных мест

Стажировка Иви-2026

Команда: Пыксели

1. Описание проекта

Название: «Парк-и-Гоу».

Проблема. В компании ограниченное число гостевых парковочных мест. Сейчас их занятость ведёт секретарь в бумажной тетради, что неудобно, непрозрачно и приводит к путанице и двойным бронированиям. Нужна система, в которой сотрудник может самостоятельно забронировать место.

Целевая аудитория. Сотрудники компании, которые приезжают на работу на машине, и администраторы офиса.

Основная ценность. Прозрачное и справедливое бронирование мест, исключение двойных броней, автоматический контроль лимитов и напоминания о предстоящих бронированиях.

2. Роли и права

В системе три роли. Доступ к данным разграничивается в зависимости от роли:

- **Гость** (неавторизованный) – просматривает список бронирований и детальные страницы, может зарегистрироваться.
- **Пользователь** (авторизованный) – всё, что доступно гостю, плюс создание броней, редактирование, отмена и удаление только своих броней, получение напоминаний.

- **Админ** (суперпользователь) – через админ-панель управляет парковочными местами, любыми бронями и пользователями.

Возможность	Гость	Пользователь	Админ
Просмотр списка и детальной страницы	+	+	+
Создание брони	-	+	+
Редактирование и отмена своей брони	-	+	+
Удаление своей брони	-	+	+
Действия с чужими бронями	-	-	+
Управление парковочными местами	-	-	+
Доступ в админ-панель	-	-	+

Ключевое ограничение доступа: пользователь работает только со своими бронями – редактировать, отменять или удалять чужие брони нельзя ни через интерфейс, ни каким-либо иным способом. Просмотр списка и детальных страниц при этом открыт всем, включая гостя, что обеспечивает прозрачность занятости.

3. Сценарии использования

Гость (неавторизованный):

- Просмотр списка бронирований с пагинацией (5–10 записей на страницу).
- Поиск по списку по одному полю (номер места или дата).
- Просмотр детальной страницы брони.
- Регистрация.

Пользователь (авторизованный):

- Вход и выход.
- Создание брони: выбор даты, свободного слота и парковочного места.
- Редактирование своей брони (с повторной проверкой пересечений и лимитов).
- Отмена своей брони – бронь помечается отменённой, слот освобождается.
- Удаление своей брони.
- Получение письма-напоминания за день до начала брони.

Администратор:

- Управление парковочными местами через админ-панель (добавление, деактивация).
- Просмотр и редактирование любых броней и пользователей.
- Доступ ко всем данным системы.

4. Бизнес-логика бронирования

Это ключевая часть проекта. Ниже описаны правила, по которым работает бронирование.

- **Слоты.** Бронирование ведётся по часовым слотам в окне с 08:00 до 22:00 (границы окна настраиваются админом). Минимальная единица брони – один час.
- **Свободные слоты не хранятся в базе.** Они вычисляются на лету: система берёт все активные брони выбранного места на выбранную дату и показывает, какие часы заняты, а какие свободны.
- **Запрет пересечений.** Новая бронь не может пересекаться по времени с другой активной бронью того же места в тот же день.

- **Лимит на день.** Один сотрудник может забронировать не более 8 часов в сутки.
- **Лимит на неделю.** Один сотрудник может забронировать не более 16 часов в неделю.
- **Учёт отмен.** Отменённые брони не занимают слот и не учитываются в суточном и недельном лимитах.
- **Где и как проверяется.** Суточный и недельный лимиты, а также понятные сообщения об ошибках проверяются в методе `clean()` модели `Booking` (код приложения на питон). Запрет пересечения броней по одному месту дополнительно закреплён ограничением на уровне PostgreSQL (`EXCLUDE` через `gist`, расширение `btree_gist`): даже при одновременных запросах база не даст создать две пересекающиеся брони на одно место.
- **Повторная проверка при редактировании.** При изменении брони проверки пересечений и лимитов выполняются заново.
- **Поведение при ошибке.** Если слот занят, интервал пересекается с существующей бронью или превышен лимит – бронь не создаётся, форма повторно показывается с сообщением об ошибке под соответствующим полем. Эти случаи покрыты тестами форм (раздел 8).
- **Напоминания.** При создании брони создаётся связанная запись `Reminder` со временем отправки за сутки до начала. Раз в день `management`-команда `send_reminders` отбирает неотправленные напоминания, срок которых наступил, и отправляет письмо сотруднику с просьбой отменить бронь, если она неактуальна. Команду по расписанию запускает планировщик.
- **Отмена и статусы.** У брони есть статус: активна или отменена. Отмена реализуется сменой статуса, а не удалением – бронь сохраняется в

истории, а слот освобождается. Полное удаление брони владельцем доступно отдельно (DeleteView).

5. Объём MVP и усложнения

В рамках MVP реализуется вся бизнес-логика из раздела 4, включая выбор конкретного парковочного места (модель хранит брони по местам, поэтому выбор места заложен в основной сценарий и не является отдельной доработкой).

Следующие усложнения берутся в работу при наличии времени и на оценку базового MVP не влияют:

- Календарь на месяц с цветовой индикацией занятости (красный – занято, зелёный – свободно, жёлтый – частично занято).
- Экспорт отчёта по занятости парковок за неделю для администратора.
- Восстановление пароля по email.

6. Нефункциональные требования

- Python 3.10+, Django 4.2+.
- База данных: PostgreSQL (используется расширение btree_gist для ограничения пересечений).
- Фронтенд: Bootstrap 5, наследование шаблонов (base.html с навигацией и футером).
- Тесты: unittest или pytest, покрытие кода не менее 70%.
- Контейнеризация: Docker Compose (web + PostgreSQL + maildev для писем в dev-среде).
- Линтеры и стиль: black, flake8, isort, .editorconfig; одинарные кавычки, f-strings, абсолютные импорты.

- Git: отдельный репозиторий, 10+ осмысленных коммитов на участника, .gitignore для Python/Django.
- Документация: README.md и .env.example.

7. Схема базы данных

В проекте четыре модели: кастомный пользователь, парковочные места, брони (основная сущность) и напоминания. Используются два типа связей: ForeignKey (бронь -> пользователь и бронь -> место) и OneToOne (напоминание -> бронь).

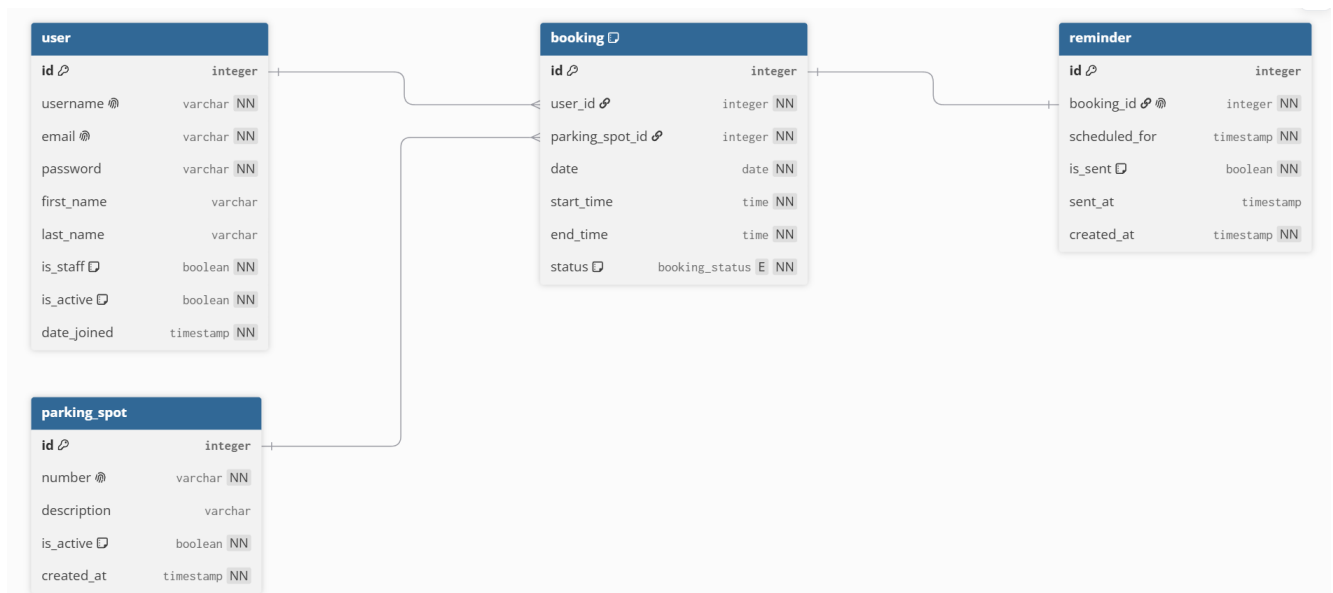


Таблица	Поля	Связи
user (кастомная)	id, username, email, password, first_name, last_name, is_staff, is_active, date_joined	—
parking_spot	id, number, description, is_active, created_at	—
booking (основная сущность)	id, user_id, parking_spot_id, date, start_time, end_time, status	user_id -> user (FK); parking_spot_id -> parking_spot (FK)

Таблица	Поля	Связи
reminder	id, booking_id, scheduled_for, is_sent, sent_at, created_at	booking_id -> booking (OneToOne)

Примечания к таблицам:

- Отдельной таблицы слотов нет: свободные слоты вычисляются из существующих броней.
- В booking дата и время (start_time, end_time) хранятся отдельными полями – это удобно для подсчёта лимитов по дню и неделе и для будущего календаря.
- Поле booking.status принимает значения «активна» / «отменена»; отменённые брони не занимают слот и не учитываются в лимитах.
- Пересечение активных броней по одному месту запрещено на уровне БД ограничением EXCLUDE (gist, расширение btree_gist) – защита от двойного бронирования при одновременных запросах. Метод clean() дублирует эту проверку для понятного сообщения в форме и проверяет суточный и недельный лимиты.
- Запись reminder связана с бронью один-к-одному; поля is_sent и sent_at нужны команде рассылки, чтобы не отправлять письмо повторно.
- Кастомная логика модели Booking: property с длительностью брони и метод валидации clean.

8. Декомпозиция работ

Оценка приведена в часах. Ответственный – ведущий по задаче; роли в команде смешанные, участники помогают друг другу. Отдельно заложено время на взаимное код-ревью, отладку и исправление багов. Распределение зон ответственности:

- **Аня** – аккаунты и авторизация.
- **Рома** – парковочные места, CRUD-в्यूхи, админка.
- **Даша** – ядро бронирования: модели, расчёт слотов, лимиты, ограничение БД.
- **Вика** – фронтенд и шаблоны.
- **Миша** – инфраструктура, напоминания, поиск, тесты, документация.

Задача	Часы	Ответственный
1. Инфраструктура и подготовка		
Каркас проекта, структура приложений, settings	2	Миша
Docker Compose (web + PostgreSQL + maildev)	3	Миша
Подключение PostgreSQL, .env и .env.example	1	Миша
Линтеры (black, flake8, isort), .editorconfig, .gitignore	2	Миша
2. Модели и база данных		
Кастомный пользователь на базе AbstractUser	1	Аня
Модель ParkingSpot	1	Рома
Модель Booking (поля, статусы)	2	Даша
Модель Reminder (связь OneToOne)	1	Даша
Валидация в clean(): пересечения, лимиты дня и недели	3	Даша
Ограничение БД от пересечений (btree_gist, ExclusionConstraint) и миграция	2	Даша
Миграции и фикстуры (стартовые места)	1	Рома
3. Авторизация и права		
Регистрация: форма, вьюха, шаблон	3	Аня

Задача	Часы	Ответственный
Вход и выход	1	Аня
Разграничение прав (гость / пользователь / админ), редиректы	2	Аня
Доступ к редактированию и удалению только своих броней	2	Аня
4. CRUD бронирований		
Список броней (ListView) и пагинация	2	Рома
Детальная страница брони (DetailView)	1	Рома
Создание брони (CreateView) и расчёт свободных слотов на дату	3	Даша
Редактирование брони (UpdateView) с повторной валидацией	2	Рома
Удаление брони (DeleteView)	1	Рома
Отмена брони (смена статуса, cancelled_at)	1	Даша
5. Поиск и админка		
Поиск по одному полю (номер места или дата)	1	Миша
Админка: регистрация моделей, list_display, фильтры, поиск	2	Рома
6. Напоминания		
Создание Reminder при брони и команда send_reminders	2	Миша
Настройка почты (maildev в dev), шаблон письма	2	Миша
7. Фронтенд		
base.html и Bootstrap 5 (навигация, футер)	3	Вика
Шаблоны списка и детальной страницы	2	Вика
Шаблон формы с выводом ошибок под полями	2	Вика
Шаблоны входа и регистрации	1	Вика
Адаптивность (карточки на мобильных) и общие стили	2	Вика

Задача	Часы	Ответственный
8. Тесты		
Тесты моделей (валидация, __str__, лимиты)	2	Даша
Тесты вьюх (список, детальная, создание, доступ без логина)	2	Рома
Тесты форм (занятый слот, превышение лимита)	2	Вика
Тесты авторизации (регистрация, редиректы)	2	Аня
Настройка покрытия, прогон, добор до 70%	1	Миша
9. Ревью и отладка		
Взаимное код-ревью мерж-реквестов	3	Команда
Отладка интеграции и исправление багов	4	Команда
10. Документация		
README (развёртывание, тесты, описание моделей, скриншоты)	2	Миша
ER-диаграмма, DECOMPOSITION.md, видео-демо	1	Команда
Итого	73	

Итоги по участникам: Миша – 16 ч, Даша – 14 ч, Рома – 12 ч, Вика – 12 ч, Аня – 11 ч, командные задачи (ревью, отладка, финальная сборка) – 8 ч. Всего – 73 ч, около 6 часов в неделю на участника.

9. Этапы работы

- **Неделя 1.** Планирование (этот документ: идея, ER-диаграмма, декомпозиция). Старт инфраструктуры: каркас проекта, Docker + PostgreSQL, кастомный User.

- **Неделя 2.** Остальные модели, миграции и ограничение БД против пересечений; админка; регистрация, вход, выход и разграничение прав; базовый CRUD (список, детальная, создание); каркас фронтенда (base.html, шаблоны списка и детальной).
- **Неделя 3.** Полный CRUD (редактирование, удаление, отмена), поиск и пагинация, напоминания (команда + maildev), оставшиеся шаблоны и адаптивность, тесты до 70%, ревью и отладка, документация и видео-демо.